NumPy 详细学习讲义

从理解概念到查阅代码

数据科学导论复习配套

对应课堂学习内容与复习资料中的 NumPy 部分

本讲义整理了学习过程中已经讲解的 NumPy 函数、方法、属性及相关规则。每一部分尽量同时回答四个问题：它是什么、什么时候使用、代码怎么写、结果是什么。
示例以普通数值数组为主，并保留字符串数组、缺失值、C++ 对比和常见错误的说明。各节可以独立查阅；运行某段代码时，请先导入 NumPy，并创建该段示例使用的变量。
本讲义不包含 Pandas 和绘图课程。

2026 年 10 月

目录

1 NumPy 是什么：先认识数组 **5**

1.1 NumPy 与 ndarray . . . 5
1.2 导入 NumPy：import . . . 5
1.3 创建数组：np.array() . . . 5
1.4 为什么不直接用 Python 列表 . . . 6
1.5 输出结果：print() 与 Jupyter . . . 6

2 数组属性：维数、形状、数量和类型 **7**

2.1 ndim、shape、size、dtype . . . 7
2.2 shape 是元组 tuple . . . 7
2.3 一维与单行二维数组的区别 . . . 8
2.4 属性和方法为什么有的加括号、有的不加 . . . 8

3 创建有规律的数组 **8**

3.1 np.arange()：指定步长 . . . 8
3.2 np.linspace()：指定数量 . . . 9

4 形状转换：reshape、转置和展平 **10**

4.1 reshape()：保持元素总数，改变排列形状 . . . 10
4.2 reshape() 中的 -1 . . . 11
4.3 转置：.T . . . 11
4.4 flatten() 与 ravel()：展平成一维 . . . 12

5 指定形状的数组：zeros、ones、empty **13**

5.1 np.zeros()：所有元素初始化为 0 . . . 13
5.2 np.ones()：所有元素初始化为 1 . . . 13
5.3 np.empty()：分配空间，但不初始化数值 . . . 13
5.4 与 C++ 初始化的对比 . . . 14

6 数据类型与 astype() **14**

6.1 dtype 的含义 . . . 14
6.2 普通数组使用统一 dtype . . . 14
6.3 astype()：转换类型 . . . 15

7 索引与切片：怎样取出数据 **15**

7.1 一维索引 . . . 15
7.2 二维索引：逗号分开行和列 . . . 15
7.3 切片：左闭右开 . . . 16
7.4 步长与倒序 . . . 17
7.5 整数索引会降维，切片通常保留维度 . . . 17
7.6 指定多个位置 . . . 18
7.7 np.ix_()：选行列交叉形成的子矩阵 . . . 18

8 逐元素运算与矩阵乘法 18
8.1 逐元素运算 . . . 18
8.2 一维数组长度不同怎么办 . . . 19
8.3 矩阵乘法：@ . . . 20

9 广播：怎样把不同形状的数组一起计算 21
9.1 标量广播 . . . 21
9.2 给二维数组每一行加同一个向量 . . . 21
9.3 给每一行加不同的标量 . . . 21
9.4 广播规则 . . . 22

10 统计与 axis 22
10.1 sum、mean、min、max . . . 22
10.2 二维数组中的 axis . . . 22
10.3 keepdims=True：保留被统计轴 . . . 24
10.4 std()：标准差 . . . 24
10.5 ddof：控制方差分母 . . . 24
10.6 median()：中位数 . . . 25
10.7 quantile()：分位数 . . . 25
10.8 cumsum()：累计和，也就是前缀和 . . . 25

11 布尔筛选、条件赋值和逻辑判断 26
11.1 比较产生布尔数组 . . . 26
11.2 布尔索引为何 True 保留、False 舍弃 . . . 26
11.3 多个条件：&、|、~ . . . 26
11.4 条件赋值：直接修改选中的位置 . . . 27
11.5 any() 与 all() . . . 27

11.6 布尔求和：统计满足条件的数量 27

12 np.where() 与最大值的位置 28

12.1 where 的三参数形式：逐个位置选择值 28
12.2 where 与筛选的区别 28
12.3 学习过程中出现的错误 29
12.4 where 的单参数形式：查找位置 29
12.5 二维 where 返回行下标和列下标 29
12.6 argmax() 与 argmin() 30

13 排序与 argsort() 31

13.1 np.sort()：返回新数组 31
13.2 a.sort()：原地排序，返回 None 31
13.3 降序排序 31
13.4 np.argsort()：返回排序顺序对应的下标 31

14 视图、别名与副本 32

14.1 b = a：同一个对象的两个名字 32
14.2 基本切片：视图 view 32
14.3 copy()：创建独立的数据副本 33
14.4 布尔索引、整数数组索引：返回副本 33

15 去重、计数与集合运算 33

15.1 np.unique()：去重并排序 33
15.2 return_counts=True：同时计数 34
15.3 交集、并集和差集 34

16 缺失值 NaN 35

16.1 NaN 是什么 35
16.2 普通统计与忽略缺失统计 35
16.3 np.isnan()：判断是否缺失 35
16.4 去掉缺失值的代码如何读 35

17 插入、追加、删除与拼接 36

17.1 np.insert()：在指定位置前插入 36
17.2 np.append()：在末尾追加 37

17.3 np.delete()：按位置删除 . . . 37
17.4 np.concatenate()：拼接数组 . . . 38

18 数学函数与随机数组 39
18.1 通用函数 ufunc . . . 39
18.2 np.random.randn()：标准正态随机数 . . . 39
18.3 default_rng() 与 integers() . . . 40

19 保存、读取与查询帮助 40
19.1 np.save()：保存数组 . . . 40
19.2 np.load()：读取数组 . . . 41
19.3 文本文件：savetxt() 与 loadtxt() . . . 41
19.4 help()：查询函数参数与示例 . . . 41

20 综合题：把已学操作串起来 42
20.1 Smith 同学成绩的平均值与标准差 . . . 42
20.2 模拟题型：创建与二维切片 . . . 43
20.3 模拟题型：利润、平均值条件与最大值位置 . . . 43
20.4 模拟题型：广播后逐行统计 . . . 44

21 查阅索引：从需求找到函数 44

22 自测练习与参考答案 46
22.1 练习 . . . 46
22.2 参考答案 . . . 46

材料范围说明 48

1 NumPy 是什么：先认识数组

1.1 NumPy 与 ndarray

NumPy 是 Python 的数值计算库，主要用于存储数组，以及高效地对一批数据进行计算。

NumPy 最核心的数组类型叫 `ndarray`，可以理解为“多维数组”。普通 ndarray 的所有元素使用统一的 `dtype`，也就是统一的元素数据类型。

一维数组可以理解成一排数；二维数组可以理解成有行和列的表格；三维数组可以理解成多张二维表叠起来。

关键点：维数与元素数量不同。一排数据有一万个元素，仍然可以是一维数组。三行两列的表格仍是二维数组，不是三维数组。

1.2 导入 NumPy：import

```
import numpy as np
```

这句代码导入 NumPy，并将它命名为 `np`。后面写 `np.array()`，就是调用 NumPy 中的 `array()` 函数。

`np` 是约定俗成的简称，不是特殊关键字。学习与考试时统一使用它，可以减少阅读和书写负担。

1.3 创建数组：np.array()

用途：把列表等数据转换成 NumPy 数组。

常用写法：

```
np.array(data)
np.array(data, dtype=int)
np.array(data, dtype=float)
```

创建一维数组：

```
a = np.array([10, 20, 30])
print(a)
```

输出：

```
[10 20 30]
```

创建二维数组：

```
data = np.array([
    [1, 2],
    [3, 4],
    [5, 6]
])
```

```
print(data)
```

输出：

```
[[1 2]
 [3 4]
 [5 6]]
```

外层列表装着整个数组，内层的每个列表提供一行数据。这个数组有 3 行、2 列。

易错点：各行之间必须有逗号，并且需要外层列表。`np.array([1,2][3,4][5,6])` 不表示三行数据；后面的方括号会被解释成索引操作。

创建普通二维数值数组时，各行长度应相同。不要期待长度不一致的嵌套列表自动补齐成矩形。

1.4 为什么不直接用 Python 列表

```
scores = [70, 80, 90]
```

Python 列表不能直接用 `scores + 5` 实现每个元素加 5。列表加列表表示拼接，列表乘整数表示重复。

NumPy 数组支持逐元素运算：

```
scores = np.array([70, 80, 90])
new_scores = scores + 5
print(new_scores)
```

输出：

```
[75 85 95]
```

这种不需要自己逐个写循环、直接对一批元素表达运算的方式，称为向量化。它通常能让数值计算代码更简洁、更高效。

1.5 输出结果：print() 与 Jupyter

在 Jupyter Notebook 中，单元格最后一个表达式的值通常会自动显示：

```
discounted = np.array([8.0, 16.0, 24.0])
discounted
```

通常显示：

```
array([ 8., 16., 24.])
```

在普通 `.py` 文件中，单独写变量名不会显示结果，应使用：

```
print(discounted)
```

输出：

```
[ 8. 16. 24.]
```

`print()` 在 Jupyter 中也能使用。赋值是“保存结果”，输出是“把结果显示出来”，两者不同。

易错点：变量名必须一致。`dicounted` 与 `discounted` 是不同名字。未定义的变量会引发 `NameError`。

2 数组属性：维数、形状、数量和类型

2.1 ndim、shape、size、dtype

```
data = np.array([
    [1, 2],
    [3, 4],
    [5, 6]
])
```

写法	含义	示例结果
`data.ndim`	数组有几个维度	`2`
`data.shape`	每个维度有多长	`(3, 2)`
`data.size`	元素总数	`6`
`data.dtype`	元素的数据类型	例如 `int64`

关键点：这四个是属性，不加调用括号。维数属性叫 `ndim`，不能简写成 `dim`。

2.2 shape 是元组 tuple

`data.shape` 的类型是元组，表示按顺序排列的一组尺寸：

```
print(data.shape)
print(type(data.shape))
```

输出：

```
(3, 2)
<class 'tuple'>
```

对于二维数组，`(3, 2)` 明确表示 **3 行、2 列**。可以用下标单独取出：

```
print(data.shape[0])
print(data.shape[1])
```

输出：

```
3
2
```

`shape[0]` 是行数，`shape[1]` 是列数。Python 的位置下标从 0 开始。

元组与 C++ 普通数组都能用下标读取元素。但 Python 元组可以混合类型，创建后不能直接替换其中的元素；C++ 普通数组的元素类型统一，非 const 数组通常可以修改元素。

2.3 一维与单行二维数组的区别

```
a = np.array([1, 2, 3])
b = np.array([[1, 2, 3]])

print(a.ndim, a.shape)
print(b.ndim, b.shape)
```

输出：

```
1 (3,)
2 (1, 3)
```

`(3,)` 表示一个维度、长度为 3；`(1, 3)` 表示两个维度、1 行 3 列。元组中只有一个元素时，末尾的逗号是正常写法。

2.4 属性和方法为什么有的加括号、有的不加

属性是直接读取的信息；方法是需要调用的功能。这取决于库的接口定义，不能照搬 C++ 的写法。

```
a.size
a.shape
a.mean()
a.sum()
```

C++ 的 `vector.size()` 是成员函数；NumPy 的 `a.size` 是属性。

易错点：对 NumPy 数组写 `a.size()` 会报 `TypeError`，因为 `a.size` 已经是一个整数，再加括号相当于试图调用这个整数。

3 创建有规律的数组

3.1 np.arange()：指定步长

用途：生成等间隔的数字序列。

写法：

```
np.arange(stop)
np.arange(start, stop)
np.arange(start, stop, step)
```

- `start`：起点；省略时为 0。
- `stop`：终点，不包含在结果中。
- `step`：步长；省略时为 1。

```
print(np.arange(5))
print(np.arange(1, 6))
print(np.arange(2, 11, 2))
```

输出：

```
[0 1 2 3 4]
[1 2 3 4 5]
[ 2  4  6  8 10]
```

创建 1 到 10，包含 10：

```
x = np.arange(1, 11)
```

创建 3、6、9、12、15：

```
y = np.arange(3, 16, 3)
```

这里把终点写成 17 也会得到同样结果：15 之后的下一个数是 18，它已经超过终点。

关键点：第三个参数是步长，不是生成几个数。整数序列通常适合使用 `arange()`；浮点等间隔序列常可使用下一节的 `linspace()`。

3.2 np.linspace()：指定数量

用途：在指定范围内生成指定数量的等间隔数值。

```
np.linspace(start, stop, num)
```

- `start`：起点。
- `stop`：终点，默认包含。
- `num`：生成的元素数量。

```
x = np.linspace(0, 10, 5)
print(x)
```

输出：

```
[ 0.   2.5  5.   7.5 10. ]
```

5 个点之间有 4 个间隔，因此间隔为：

$$\frac{10-0}{5-1}=2.5.$$

```
print(np.linspace(0, 1, 5))
```

输出：

```
[0.   0.25 0.5  0.75 1.  ]
```

函数	第三个参数	终点规则
`arange()`	步长	不包含终点
`linspace()`	元素数量	默认包含终点

需要排除终点时，可以显式指定：

```
np.linspace(0, 1, 5, endpoint=False)
```

4 形状转换：reshape、转置和展平

4.1 reshape()：保持元素总数，改变排列形状

```
a = np.arange(1, 7)
b = a.reshape(2, 3)
print(b)
```

输出：

```
[[1 2 3]
 [4 5 6]]
```

`reshape(2, 3)` 表示排列成 2 行、3 列。默认按行顺序读取并重新放置元素。

创建 1 到 12，并直接排列成 3 行、4 列：

```
matrix = np.arange(1, 13).reshape(3, 4)
```

结果：

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

关键点：变形前后元素总数必须相同：行数 × 列数 = 元素总数。函数不会自动补齐或丢弃元素。

4.2 reshape() 中的 -1

`-1` 表示“这个维度的长度由 NumPy 自动推算”。

```
a = np.arange(1, 13)

a.reshape(3, -1)
a.reshape(-1, 4)
```

两者都得到 3 行、4 列，因为一共 12 个元素。

```
a.reshape(2, -1)
```

得到 2 行、6 列，形状为 `(2, 6)`。

以下会报错：

```
a.reshape(5, -1)
a.reshape(-1, -1)
```

第一个无法整除；第二个有两个未知尺寸。最多只能有一个维度写成 `-1`。

易错点：`-1` 不能换成冒号。`reshape(2, :)` 是语法错误。`-1` 在 reshape 中推算尺寸，`:` 在方括号索引中指定选取范围，作用不同。

`reshape()` 返回变形后的数组，不会仅靠调用就改变原变量所表示的形状；需要结果时应赋值。返回结果可能与原数组共享内存，不能假定一定是独立副本。

4.3 转置：.T

二维数组转置就是交换行和列。

```
a = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

b = a.T
print(b)
```

输出：

```
[[1 4]
 [2 5]
 [3 6]]
```

原数组是 2 行、3 列；转置后是 3 行、2 列。

关键点：`.T` 是属性，不写成 `.T()`。

转置与 reshape 不同：

```
print(a.reshape(3, 2))
```

输出：

```
[[1 2]
 [3 4]
 [5 6]]
```

两者形状相同，但元素对应关系不同。转置按“行变列”的规则，reshape 按元素顺序重新排列。

普通 ndarray 的转置通常共享原数组内存；需要独立结果时写 `a.T.copy()`。一维数组没有二维的行列可交换，所以仅写 `v.T` 不会把一维数组变成二维列向量。

4.4 flatten() 与 ravel()：展平成一维

```
a = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

b = a.flatten()
c = a.ravel()
```

两者数值结果都是：

```
[1 2 3 4 5 6]
```

- `flatten()`：总是返回副本。
- `ravel()`：能共享内存时共享，否则返回副本。

```
b[0] = 99
print(a[0, 0])
```

输出仍然是 `1`，因为 `b` 是副本。

如果需要确保独立，可以用：

```
b = a.flatten()
c = a.ravel().copy()
```

5 指定形状的数组：zeros、ones、empty

5.1 np.zeros()：所有元素初始化为 0

```
z = np.zeros((3, 4), dtype=int)
print(z)
```

输出：

```
[[0 0 0 0]
 [0 0 0 0]
 [0 0 0 0]]
```

(3, 4) 是一个形状元组，表示 3 行、4 列。外层括号调用函数，内层括号构成元组。

默认数据类型是浮点数：

```
np.zeros((2, 3))
```

结果中的零会显示成 0.。

一维数组：

```
np.zeros(5, dtype=int)
```

5.2 np.ones()：所有元素初始化为 1

```
w = np.ones((2, 4), dtype=int)
print(w)
```

输出：

```
[[1 1 1 1]
 [1 1 1 1]]
```

参数结构与 zeros() 相同。

5.3 np.empty()：分配空间，但不初始化数值

```
e = np.empty((3, 2), dtype=int)
```

它已经是一个 3 行、2 列的数组，但里面的数值不确定。不要假定是 0，也不要拿未初始化的值直接计算。

应先填入数据：

```
e[:] = 0
e[:, 1] = 9
```

```
print(e)
```

输出：

```
[[0 9]
 [0 9]
 [0 9]]
```

e[:] 选取所有行，因此覆盖整个二维数组；e[:, 1] 选所有行中的第二列。

5.4 与 C++ 初始化的对比

vector<int> v(6) 会将 6 个整数初始化为 0，因此更接近：

```
v = np.zeros(6, dtype=int)
```

np.empty(6, dtype=int) 的“数值未初始化”特征，更接近 C++ 函数内部未初始化的普通局部数组 int v[6];。

这是帮助理解初始化行为的类比，不表示底层实现完全相同。

6 数据类型与 astype()

6.1 dtype 的含义

dtype 表示数组元素的数据类型。常用类型有整数、浮点数、布尔值和字符串。

```
a = np.array([1, 2, 3])
b = np.array([1.5, 2.8, 3.0])

print(a.dtype)
print(b.dtype)
```

整数类型显示可能随环境而不同；浮点数通常为 float64。课件还提到了：

- np.uint8：8 位无符号整数，范围为 0 到 255。
- np.int64：64 位有符号整数。
- np.float32：32 位浮点数。
- np.float64：64 位浮点数。

6.2 普通数组使用统一 dtype

```
c = np.array([1, 2.5, 3])
print(c)
```

输出：

```
[1.  2.5 3. ]
```

为了容纳小数，整数也转换成了浮点数。

如果把字符串和数字混在一起，可能统一转换成字符串。这与 Python 列表可以分别保留各元素类型不同。

6.3 astype()：转换类型

```
x = np.array([2, 4, 6])
y = x.astype(float)
print(y)
```

输出：

```
[2. 4. 6.]
```

`astype()` 返回转换后的数组。默认情况下不会把原数组 `x` 改成浮点数组。

字符串数字也可以转换：

```
s = np.array(["12", "13", "14"])
values = s.astype(int)
```

结果为整数数组 `[12 13 14]`。

易错点：无法表示为指定类型的字符串会导致转换失败，例如 `"hello"` 不能直接转换成整数。浮点数转整数通常截去小数部分，不是四舍五入。

7 索引与切片：怎样取出数据

7.1 一维索引

```
a = np.array([10, 20, 30, 40, 50])

a[0]
a[2]
a[-1]
```

分别得到 10、30、50。

正下标从 0 开始；`-1` 是最后一个，`-2` 是倒数第二个。

7.2 二维索引：逗号分开行和列

```
data = np.array([
```

```
    [1, 2],
    [3, 4],
    [5, 6]
])
```

固定结构：

```
data[row_index, column_index]
```

例如：

```
data[2, 1]
```

得到 6，因为它在第三行、第二列。

```
data[0, :]
data[:, 1]
```

分别得到：

```
[1 2]
[2 4 6]
```

`:` 表示该方向全部选取。

关键点：只写一个索引时，默认作用于第一维，也就是二维数组的行。`data[1]` 取第二行，不是第二列。选第二列应写 `data[:, 1]`。

C++ 普通二维数组通常写 `a[1][0]`；NumPy 常用 `a[1, 0]`，在同一对方括号里分开行列。

7.3 切片：左闭右开

```
a = np.array([10, 20, 30, 40, 50])

a[1:4]
a[:2]
a[1:]
```

分别得到：

```
[20 30 40]
[10 20]
[20 30 40 50]
```

`start:stop` 包含开始位置，不包含结束位置。

二维切片可以同时指定行列：

```
b = np.array([
    [10, 20, 30],
```

```
    [40, 50, 60],
    [70, 80, 90]
])

b[1:, :2]
```

输出：

```
[[40 50]
 [70 80]]
```

行选择第二行到最后，列选择前两列。

7.4 步长与倒序

```
a = np.array([10, 20, 30, 40, 50, 60])

a[::2]
a[1::2]
a[::-1]
```

分别得到：

```
[10 30 50]
[20 40 60]
[60 50 40 30 20 10]
```

完整格式是：

```
a[start:stop:step]
```

`a[1::2]` 从下标 1 开始，每次前进 2 个位置。`a[::-1]` 从末尾开始倒着逐个读取。

7.5 整数索引会降维，切片通常保留维度

```
data = np.array([
    [1, 2],
    [3, 4],
    [5, 6]
])

print(data[:, 0].shape)
print(data[:, 0:1].shape)
```

输出：

```
(3,)
```

```
(3, 1)
```

`data[:, 0]` 是一维数组；`data[:, 0:1]` 是 3 行、1 列的二维数组。

类似地，`data[1]` 是一维的一行，而 `data[1:2, :]` 是 1 行、2 列的二维数组。

7.6 指定多个位置

```
a = np.array([10, 20, 30, 40])
a[[0, 2]]
```

得到 `[10 30]`。内部列表提供需要选取的下标。

二维整数数组索引会配对：

```
a = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])

a[[0, 2], [1, 2]]
```

得到 `[2 9]`，选的是两个坐标：

$$(\text{行}0, \text{列}1), \quad (\text{行}2, \text{列}2).$$

7.7 np.ix_()：选行列交叉形成的子矩阵

如果要第一行、第三行与第二列、第三列的所有组合：

```
sub = a[np.ix_([0, 2], [1, 2])]
print(sub)
```

输出：

```
[[2 3]
 [8 9]]
```

关键点：双列表索引是配对坐标；`np.ix_()` 用于构造行列的交叉组合。

8 逐元素运算与矩阵乘法

8.1 逐元素运算

两个兼容形状的数组，可以让对应元素进行计算：

```
a = np.array([10, 20, 30])
```

```
b = np.array([1, 2, 3])

print(a + b)
print(a * b)
```

输出：

```
[11 22 33]
[10 40 90]
```

常见运算符：

- `+`、`-`：逐元素加减。
- `*`、`/`：逐元素乘除。
- `**`：逐元素幂运算。
- `%`：逐元素取余。

商品打八折：

```
prices = np.array([10, 20, 30])
discounted = prices * 0.8
```

利润计算：

```
prices = np.array([12, 20, 30])
costs = np.array([7, 14, 18])
qty = np.array([2, 3, 4])

profit = (prices - costs) * qty
total_profit = profit.sum()
```

结果：

```
profit: [10 18 48]
total_profit: 76
```

8.2 一维数组长度不同怎么办

```
np.array([10, 20, 30]) * np.array([2])
```

得到 `[20 40 60]`，长度为 1 的数组可以广播。

但：

```
np.array([10, 20, 30]) * np.array([2, 3])
```

会报错，因为长度 3 和 2 不兼容。不会自动补齐，也不会只计算前两个。

商品数据必须一一对应。广播规则允许计算，并不自动保证业务上的对应关系正确。

8.3 矩阵乘法：@

`A * B` 是逐元素乘法；`A @ B` 是矩阵乘法。

```
A = np.array([
    [1, 2],
    [3, 4]
])

B = np.array([
    [5, 6],
    [7, 8]
])

print(A * B)
print(A @ B)
```

输出：

```
[[ 5 12]
 [21 32]]

[[19 22]
 [43 50]]
```

矩阵乘法的每个结果元素，是左边的一行与右边的一列，对应相乘后求和。例如左上角：

$$1 \times 5 + 2 \times 7 = 19.$$

四个元素分别为：

$$C_{00} = 1 \times 5 + 2 \times 7 = 19,$$
$$C_{01} = 1 \times 6 + 2 \times 8 = 22,$$
$$C_{10} = 3 \times 5 + 4 \times 7 = 43,$$
$$C_{11} = 3 \times 6 + 4 \times 8 = 50.$$

关键点：二维矩阵乘法要求：左边的列数等于右边的行数。

一个 3 行、2 列的数组乘一个 2 行、5 列的数组：

$$(3, 2) @ (2, 5) \longrightarrow (3, 5).$$

左边每行有 2 个元素，右边每列也有 2 个元素，因此可以一一配对。结果有 3 行、5 列。

两个二维数组也可用 `np.dot(A, B)` 做矩阵乘法。对更高维数组，`dot` 与 `@` 的规则有区别；本讲义先掌握二维情况。

9 广播：怎样把不同形状的数组一起计算

9.1 标量广播

```
a = np.array([10, 20, 30])
a + 5
```

得到 `[15 25 35]`。标量 5 用于每个元素。

9.2 给二维数组每一行加同一个向量

```
a = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

offset = np.array([10, 20, 30])
b = a + offset
print(b)
```

输出：

```
[[11 22 33]
 [14 25 36]]
```

`a` 是 2 行、3 列，形状 `(2, 3)`。`offset` 是一维 3 个元素，形状 `(3,)`。

每一行都加上同一个向量，也可以从列的角度理解为：第一列加 10、第二列加 20、第三列加 30。

9.3 给每一行加不同的标量

第一行全部加 10，第二行全部加 20：

```
offset = np.array([10, 20]).reshape(2, 1)
b = a + offset
print(b)
```

输出：

```
[[11 12 13]
 [24 25 26]]
```

偏移数组是 2 行、1 列，形状 `(2, 1)`。每行只有一个偏移值，广播到该行的全部列。

复习资料还使用了：

```
offset = np.array([10, 20])
```

```
column_offset = offset[:, None]
```

这里 None 新增一个轴，把形状 (2,) 变成 (2, 1)。对这个例子，它与 reshape(2, 1) 效果相同。

9.4 广播规则

从形状的最右端开始对齐，每对维度需要：

1. 长度相同；或者
2. 其中一个长度为 1。

缺少的前置维度按长度为 1 看待。

对于一个 2 行、3 列的数组：

偏移形状	能否广播	效果
(3,)	可以	三列分别加偏移
(2, 1)	可以	两行分别加偏移
(2,)	不可以	右端 3 与 2 不匹配

易错点：广播不是“元素总数一样就行”。应检查形状的对应维度，而不是只看 size。

10 统计与 axis

10.1 sum、mean、min、max

```
scores = np.array([60, 80, 100])

scores.sum()
scores.mean()
scores.min()
scores.max()
```

分别得到：

```
240
80.0
60
100
```

这些是方法，需要括号。也可写成 np.sum(scores)、np.mean(scores) 等函数形式。

10.2 二维数组中的 axis

```
scores = np.array([
    [80, 90, 70],
    [60, 80, 100]
])
```

这是 2 行、3 列：两位学生、三门科目。

```
scores.mean()
```

不指定轴，计算全部 6 个元素的平均值，得到 80.0。

```
scores.mean(axis=0)
```

得到每科平均分：

```
[70. 85. 85.]
```

```
scores.mean(axis=1)
```

得到每位学生平均分：

```
[80. 80.]
```

二维数组操作	哪些数放在一起计算	输出
axis=0	同一列，上下各行	每列一个结果
axis=1	同一行，左右各列	每行一个结果
不指定 axis	全部元素	一个结果

可以理解为：统计运算“消去”指定的那个轴。

对于形状 (m, n)：

- 沿 axis=0 统计，通常剩下 (n,)。
- 沿 axis=1 统计，通常剩下 (m,)。

```
totals = scores.sum(axis=1)
highest = scores.max(axis=0)
```

结果：

```
totals: [240 240]
highest: [80 90 100]
```

关键点：用结果数量检查方向：两位学生的总分应有两个数；三门科目的最高分应有三个数。

10.3 keepdims=True：保留被统计轴

```
a = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

row_sum = a.sum(axis=1, keepdims=True)
print(row_sum)
print(row_sum.shape)
```

输出：

```
[[ 6]
 [15]]
(2, 1)
```

默认逐行求和的结果形状是 (2,)。keepdims=True 保留长度为 1 的列轴，得到 2 行、1 列的结果，方便继续广播运算。

10.4 std()：标准差

标准差衡量数据的分散程度。[79, 80, 81] 与 [60, 80, 100] 平均值都是 80，但后者更分散，标准差更大。

```
a = np.array([79, 80, 81])
b = np.array([60, 80, 100])

print(a.std())
print(b.std())
```

分别约为 0.8165 和 16.3299。所有数相同时，标准差为 0。

10.5 ddof：控制方差分母

标准差计算为：

$$s = \sqrt{\frac{\sum_{i=1}^{N}(x_i - \bar{x})^2}{N - \text{ddof}}}.$$

- ddof=0：分母为 N，NumPy 默认值。
- ddof=1：分母为 $N-1$，常用于样本方差估计。

```
a = np.array([1, 2, 3])

a.std(ddof=0)
a.std(ddof=1)
```

平均值为 2，偏差平方和为 2，因此结果分别是：

$$\sqrt{2/3} \approx 0.8165, \qquad \sqrt{2/2} = 1.$$

用样本自身的平均值计算偏差，会使总体方差估计偏小。在常见的独立同分布抽样条件下，分母使用 $N-1$ 可以得到总体方差的无偏估计。这里的无偏结论针对方差，不意味着开平方后的标准差也严格无偏。

易错点：NumPy 的 `std()` 默认 `ddof=0`，Pandas 的 `Series.std()` 默认 `ddof=1`。比较结果时必须使用同一口径。有效样本数需要大于 ddof。

10.6 median()：中位数

数据排序后，奇数个元素取中间那个；偶数个元素取中间两个的平均值。

```
np.median([9, 1, 5])
np.median([2, 4, 8, 10])
```

分别得到 `5.0` 与 `6.0`。

10.7 quantile()：分位数

```
a = np.array([0, 10, 20, 30, 40])

np.quantile(a, 0.25)
np.quantile(a, 0.5)
np.quantile(a, 0.75)
```

分别得到 `10.0`、`20.0`、`30.0`，对应第一四分位数、中位数、第三四分位数。

一次计算多个：

```
np.quantile(a, [0.25, 0.5, 0.75])
```

输出：

```
[10. 20. 30.]
```

分位参数在 0 到 1 之间。默认采用线性插值，结果不一定是原数组中的值。`np.percentile()` 则使用 0 到 100 的百分数尺度。

10.8 cumsum()：累计和，也就是前缀和

```
daily = np.array([3, 5, 2])

daily.sum()
np.cumsum(daily)
```

分别得到：

```
10
[ 3  8 10]
```

累计和依次为 3、3 + 5、3 + 5 + 2。它不会额外在开头放一个 0。

11 布尔筛选、条件赋值和逻辑判断

11.1 比较产生布尔数组

```
scores = np.array([60, 85, 72, 90])
mask = scores >= 80
print(mask)
```

输出：

```
[False  True False  True]
```

每个位置都保存对应元素是否满足条件。

11.2 布尔索引为何 True 保留、False 舍弃

```
good_scores = scores[mask]
```

得到 `[85 90]`。

这是 NumPy 对布尔索引规定的规则：把布尔数组放到索引方括号中，它就是一张选择名单，`True` 表示选取对应位置，`False` 表示不选。

单独的 `mask` 只是一组布尔值；`scores[mask]` 才执行筛选。

关键点：筛选不会删除原数组中的数据，只是返回选中的数据。

可以写成：

```
good_scores = scores[scores >= 80]
```

11.3 多个条件：&、|、~

```
prices = np.array([12, 35, 8, 50, 20])

selected = prices[
    (prices >= 20) & (prices < 50)
]
```

得到 `[35 20]`，保留原来的相对顺序。

- `&`：并且。
- `|`：或者。
- `~`：对布尔值取反。

```
selected = prices[
    (prices < 10) | (prices > 40)
]
```

得到 `[8 50]`。

易错点：数组条件不要写 `20 <= prices <= 50`，也不要用普通 Python 的 `and`/`or` 直接组合多元素数组。应拆开比较，并给各条件加括号。

11.4 条件赋值：直接修改选中的位置

```
a = np.array([1, 2, 3, 4])
a[a > 2] = 0
print(a)
```

输出：

```
[1 2 0 0]
```

左边是原数组的索引目标，因此会修改原数组。这与先取出筛选结果、再修改结果不同，后面“视图与副本”一节会进一步解释。

11.5 any() 与 all()

```
a = np.array([2, 4, 6])

np.any(a > 5)
np.all(a > 5)
```

分别得到 `True` 与 `False`。

- `any()`：是否至少有一个条件成立。
- `all()`：是否所有条件都成立。

它们返回一个整体判断，而不是筛选后的元素数组。

11.6 布尔求和：统计满足条件的数量

```
(a > 5).sum()
```

得到整数 1。布尔值求和时，True 按 1、False 按 0 计算。

关键点：True、False 是布尔值；计数得到的 1、0 是整数。在计算中可对应，不表示输出类型完全相同。

12 np.where() 与最大值的位置

12.1 where 的三参数形式：逐个位置选择值

```
np.where(condition, x, y)
```

条件为真时取 x 对应的值，否则取 y 对应的值。

```
scores = np.array([55, 80, 60, 42])

result = np.where(
    scores >= 60,
    "pass",
    "fail"
)
```

结果：

```
['fail' 'pass' 'pass' 'fail']
```

保持及格成绩，不及格改成 0：

```
adjusted = np.where(scores >= 60, scores, 0)
```

结果：

```
[ 0 80 60  0]
```

限制价格上限为 100：

```
prices = np.array([80, 150, 60, 200])
adjusted_prices = np.where(prices <= 100, prices, 100)
```

结果：

```
[ 80 100  60 100]
```

12.2 where 与筛选的区别

```
prices = np.array([12, 35, 8, 50, 20])

prices[prices > 20]
```

```
np.where(prices > 20, 1, 0)
```

分别得到：

```
[35 50]
[0 1 0 1 0]
```

布尔筛选只返回匹配的值；三参数 where 为每个位置生成结果。在这些示例中，where 输出长度与原数组相同。

12.3 学习过程中出现的错误

- NumPy 的写法是 `np.where(...)`，普通 ndarray 没有这里使用的 `scores.where(...)`。
- 判断成绩用 `scores >= 60`，不要误用保存字符串标记的 `result`。
- `pass` 是 Python 关键字，不能作变量名；`"pass"` 是字符串，可以使用。
- 变量名在真实 Python 代码中不能用 Markdown 反引号包住。

12.4 where 的单参数形式：查找位置

```
a = np.array([3, 8, 2, 9])

positions = np.where(a > 5)[0]
print(positions)
```

输出：

```
[1 3]
```

8 与 9 的下标分别是 1、3。

为什么有 `[0]`？单参数 where 返回一个元组：

```
np.where(a > 5)
```

结果：

```
(array([1, 3]),)
```

一维数组只有一个轴，所以元组只有一项。用 `[0]` 取出这一项，即位置数组。

12.5 二维 where 返回行下标和列下标

```
a = np.array([
    [3, 8],
    [9, 2]
```

```
])

rows, cols = np.where(a > 5)
```

结果：

```
rows: [0 1]
cols: [1 0]
```

同一位置的行列下标需要配对读取：

- 第一个匹配：行 0、列 1，元素为 8。
- 第二个匹配：行 1、列 0，元素为 9。

对于一维、二维及更高维的普通数组，单参数 where 的返回元组每个轴对应一项。元组项数表示轴数，项中数组的长度表示匹配数量。

12.6 argmax() 与 argmin()

```
a = np.array([30, 10, 50, 20])

a.max()
a.argmax()
a.argmin()
```

分别得到 50、2、1。

- max()：最大值。
- argmax()：最大值的位置。
- argmin()：最小值的位置。

也可以写：

```
np.argmax(a)
np.argmin(a)
```

最大值重复时，返回第一次出现的位置：

```
profit = np.array([12, 35, 8, 35, 20])
best_idx = int(np.argmax(profit))
```

结果为 Python 整数 1。int() 可以满足题目要求“普通整数”的输出类型。

多维数组不指定轴时，argmax 默认返回展平后的下标；不能直接把它当作二维行列坐标。指定 axis=0 或 axis=1 时，则返回对应轴上的最大值位置。

13 排序与 argsort()

13.1 np.sort()：返回新数组

```
a = np.array([30, 10, 50, 20])
b = np.sort(a)

print(b)
print(a)
```

输出：

```
[10 20 30 50]
[30 10 50 20]
```

原数组不变，排序结果用新变量接收。

13.2 a.sort()：原地排序，返回 None

```
a.sort()
print(a)
```

得到：

```
[10 20 30 50]
```

易错点：不要写 `a = a.sort()`。方法已经修改了原数组，但返回值为 `None`；再次赋值会让变量 a 变成 None。

它在“直接修改原数据”这一点上，类似 C++ 的 `std::sort(v.begin(), v.end())`。C++ 的 std::sort 返回 void，NumPy 的数组 sort 返回 None。

13.3 降序排序

```
descending = np.sort(a)[::-1]
```

先升序排序，再用倒序切片，得到降序。不会修改原数组。

13.4 np.argsort()：返回排序顺序对应的下标

```
a = np.array([30, 10, 50, 20])
order = np.argsort(a)
```

结果：

```
[1 3 0 2]
```

因为升序的 10、20、30、50，在原数组中的位置是 1、3、0、2。

```
a[order]
```

得到：

```
[10 20 30 50]
```

它适合让多组对应数据按同一顺序排列：

```
names = np.array(["Wang", "Li", "Zhang"])
scores = np.array([80, 95, 70])

order = np.argsort(scores)

print(scores[order])
print(names[order])
```

输出：

```
[70 80 95]
['Zhang' 'Wang' 'Li']
```

姓名与成绩仍然一一对应。

14 视图、别名与副本

14.1 b = a：同一个对象的两个名字

```
a = np.array([10, 20, 30])
b = a
b[0] = 99
```

a 也会变成 [99 20 30]，因为没有复制数组，两个名字指向同一个对象。

14.2 基本切片：视图 view

```
a = np.array([10, 20, 30, 40])
b = a[1:3]
b[0] = 99
print(a)
```

输出：

```
[10 99 30 40]
```

基本切片得到视图，与原数组共享对应数据。b[0] 对应原数组的 a[1]。

可以把视图理解为观察原数据一部分的窗口。

14.3 copy()：创建独立的数据副本

```
a = np.array([10, 20, 30, 40])
b = a[1:3].copy()
b[0] = 99

print(b)
print(a)
```

输出：

```
[99 30]
[10 20 30 40]
```

对于本讲义中的数值数组，副本拥有独立的数据存储。

关键点：可以用“共享数据”和“复制数据”帮助理解，但准确名称是视图和副本，不宜简单把所有情况称为浅拷贝和深拷贝。如果数组存储 Python 对象，copy 不会递归复制对象内部内容。

14.4 布尔索引、整数数组索引：返回副本

```
a = np.array([1, 2, 3, 4])
b = a[a > 2]
b[0] = 99

print(a)
```

输出仍为：

```
[1 2 3 4]
```

布尔索引和整数数组索引属于高级索引，读取时返回副本。

但直接赋值：

```
a[a > 2] = 0
```

会修改原数组，得到 `[1 2 0 0]`。这是索引赋值，并不是修改一个先前取出的独立结果。

15 去重、计数与集合运算

15.1 np.unique()：去重并排序

```
a = np.array([3, 1, 3, 2, 1])
values = np.unique(a)
```

得到：

```
[1 2 3]
```

默认输出不同的值，并排序。

15.2 return_counts=True：同时计数

```
values, counts = np.unique(a, return_counts=True)
```

结果：

```
values: [1 2 3]
counts: [2 1 2]
```

两组结果一一对应：1 出现两次、2 出现一次、3 出现两次。

函数返回两个数组，左边的两个变量分别接收它们。

15.3 交集、并集和差集

```
a = np.array([1, 2, 3])
b = np.array([2, 3, 4])

np.intersect1d(a, b)
np.union1d(a, b)
np.setdiff1d(a, b)
np.setdiff1d(b, a)
```

分别得到：

```
[2 3]
[1 2 3 4]
[1]
[4]
```

- `intersect1d()`：两边都有的值。
- `union1d()`：两边所有不同的值。
- `setdiff1d(a, b)`：a 中有、b 中没有的值。

默认会去重并排序。这些函数按值处理集合，不保留原数组的二维表格结构。

16 缺失值 NaN

16.1 NaN 是什么

`np.nan` 可用于表示缺失数值，例如没有录入的成绩：

```
scores = np.array([80, np.nan, 100])
```

普通浮点数组可以存储 NaN，普通整数 dtype 不能直接表示 NaN。

16.2 普通统计与忽略缺失统计

```
scores.mean()
scores.sum()

np.nanmean(scores)
np.nansum(scores)
```

结果分别是：

```
nan
nan
90.0
180.0
```

`nanmean()` 忽略 NaN，用 80 与 100 求平均值；不是把缺失值当作 0 再平均。

16.3 np.isnan()：判断是否缺失

```
np.isnan(scores)
```

输出：

```
[False  True False]
```

不要用 `scores == np.nan` 判断，因为 NaN 不等于它自身。

16.4 去掉缺失值的代码如何读

```
valid = scores[~np.isnan(scores)]
```

按三个步骤拆解：

1. `np.isnan(scores)` 得到 `[False, True, False]`，标记缺失位置。
2. `~` 取反，得到 `[True, False, True]`，标记非缺失位置。

3. 布尔索引保留 True 对应的值，得到 `[80., 100.]`。

等价的分步写法：

```
mask = ~np.isnan(scores)
valid = scores[mask]
```

`valid` 是普通变量名，并不自带特殊功能。

17 插入、追加、删除与拼接

17.1 np.insert()：在指定位置前插入

```
a = np.array([10, 20, 30])
b = np.insert(a, 1, 99)
```

结果：

```
[10 99 20 30]
```

三个参数依次是原数组、插入位置、插入值。位置 1 表示在原来第二个元素之前插入。

二维数组插入一行：

```
a = np.array([
    [1, 2],
    [3, 4]
])

b = np.insert(a, 1, [8, 9], axis=0)
```

结果：

```
[[1 2]
 [8 9]
 [3 4]]
```

插入一列：

```
c = np.insert(a, 1, [8, 9], axis=1)
```

结果：

```
[[1 8 2]
 [3 9 4]]
```

`axis=0` 插入行，`axis=1` 插入列。

17.2 np.append()：在末尾追加

```
a = np.array([10, 20, 30])
b = np.append(a, [40, 50])
```

结果：

```
[10 20 30 40 50]
```

它返回新数组，不像 C++ 的 `vector.push_back()` 那样直接修改原容器。

```
a = np.array([1, 2])
np.append(a, 3)
print(a)
```

输出仍然是 `[1 2]`。要更新 a，必须接住结果：

```
a = np.append(a, 3)
```

二维追加行：

```
matrix = np.array([
    [1, 2],
    [3, 4]
])

b = np.append(matrix, [[5, 6]], axis=0)
```

结果：

```
[[1 2]
 [3 4]
 [5 6]]
```

易错点：二维数组使用 append 时，不指定 axis 会先展平。`np.append(matrix, [5, 6])` 得到一维的 `[1 2 3 4 5 6]`。

17.3 np.delete()：按位置删除

```
a = np.array([10, 20, 30, 40])

b = np.delete(a, 1)
c = np.delete(a, -1)
```

分别得到：

```
[10 30 40]
[10 20 30]
```

删除的是下标对应的位置，不是数值本身。

删除多处：

```
np.delete(a, [0, 2])
```

得到 `[20 40]`。

二维删除：

```
np.delete(matrix, 0, axis=0)
np.delete(matrix, 0, axis=1)
```

分别删除第一行、第一列。

关键点：insert、append、delete 都返回新数组，不会直接改变原数组。题目要求更新变量时必须赋值。

17.4 np.concatenate()：拼接数组

一维：

```
a = np.array([1, 2])
b = np.array([3, 4])
c = np.concatenate([a, b])
```

结果：

```
[1 2 3 4]
```

要拼接的数组放在列表 `[a, b]` 中，作为一个参数传入。

二维上下拼接：

```
a = np.array([
    [1, 2],
    [3, 4]
])
b = np.array([[5, 6]])

c = np.concatenate([a, b], axis=0)
```

得到 3 行、2 列：

```
[[1 2]
 [3 4]
 [5 6]]
```

轴	二维效果	需要满足
axis=0	上下拼，增加行数	列数相同
axis=1	左右拼，增加列数	行数相同

一般规则是：除了拼接的轴，其他维度都要匹配。

18 数学函数与随机数组

18.1 通用函数 ufunc

通用函数可以对数组中每个元素分别做同一种计算。

```
a = np.array([1, 4, 9, 16])
roots = np.sqrt(a)
```

结果：

```
[1. 2. 3. 4.]
```

常见示例：

```
np.abs(np.array([-3, 0, 2]))
np.exp(np.array([0, 1]))
np.log(np.array([1, np.e]))
```

分别是绝对值、以 e 为底的指数、自然对数：

```
[3 0 2]
[1.         2.71828183]
[0. 1.]
```

sqrt() 等函数分别计算每个元素，而 sum() 将多个元素汇总。

复习资料还列出：

```
np.sin(np.array([0, np.pi / 2]))
np.cos(np.array([0, np.pi]))
np.round(np.array([1.234, 2.678]), 2)
```

正弦与余弦的输入单位是弧度。round(..., 2) 表示保留两位小数；题目未要求舍入时，不要主动舍入。

18.2 np.random.randn()：标准正态随机数

```
r = np.random.randn(3, 4)
```

得到 3 行、4 列的数组。数值从标准正态分布抽取，分布均值为 0、标准差为 1。

大多数数在 0 附近，正负都可能出现；单次小样本的实际平均值不必恰好为 0。每次运行通常得到不同数值。

易错点：`randn(3, 4)` 的维度分别传入；`zeros((3, 4))` 的形状作为一个元组传入。不同函数的参数接口不能完全照搬。

18.3 default_rng() 与 integers()

```
rng = np.random.default_rng(42)
a = rng.integers(0, 10, size=(2, 3))
```

- `42` 是随机种子。
- `0` 是下界，包含。
- `10` 是上界，默认不包含。
- `size=(2, 3)` 表示 2 行、3 列。

生成的是 0 到 9 的整数。

重新创建相同种子的生成器，可以复现相同的随机序列。但在同一个生成器上连续调用，随机状态会前进，不会每次都输出同一组数。

生成标准正态数组：

```
rng = np.random.default_rng(42)
r = rng.standard_normal((2, 3))
```

这是复习资料中使用的生成器接口；课件中的 `randn()` 也需要认识。

19 保存、读取与查询帮助

19.1 np.save()：保存数组

```
a = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

np.save("my_array.npy", a)
```

第一个参数是文件名，第二个参数是数组。`.npy` 格式保留数值、形状和数据类型。

只写文件名，文件保存到 Python 当前工作目录；也可以写完整路径。同名文件会被覆盖。

19.2 np.load()：读取数组

```
b = np.load("my_array.npy")
print(b)
```

输出：

```
[[1 2 3]
 [4 5 6]]
```

读取函数返回数组，用变量接收。对不可信文件，不要随意启用对象反序列化选项。

19.3 文本文件：savetxt() 与 loadtxt()

复习资料还列出了数值数组的文本读写：

```
np.savetxt(
    "array.csv",
    a,
    delimiter=",",
    fmt="%d"
)

b = np.loadtxt(
    "array.csv",
    delimiter=",",
    dtype=int
)
```

`delimiter` 是分隔符；`fmt="\%d"` 表示按整数格式写出。`savetxt()` 主要适用于一维、二维数值数组。复杂表头、混合类型表格通常更适合用 Pandas。

19.4 help()：查询函数参数与示例

```
help(np.arange)
```

这里把函数本身交给 help，不是先调用它。

易错点：不要写 `help(np.arange())`；这会先尝试运行 arange，而不是直接查看说明。

查询数组方法：

```
help(np.ndarray.reshape)
help(np.ndarray.std)
```

Jupyter 中也可以写：

```
np.arange?
```

这是 Jupyter 的帮助语法，不能直接当作普通 `.py` 文件中的 Python 语句。

`dir(np)` 列出 NumPy 中的名称：

```
dir(np)
```

Jupyter 中输入 `np.` 后按 Tab，也可查看补全候选。

官方文档：https://numpy.org/doc/stable/reference/

阅读函数页面时，重点看 Parameters、Returns、Examples：分别是参数、返回结果与示例。

20 综合题：把已学操作串起来

20.1 Smith 同学成绩的平均值与标准差

```
data = np.array([
    ["Bob",  "Smith",  "12"],
    ["Joe",  "Smith",  "13"],
    ["Roy",  "Ratner", "12"],
    ["Rita", "Smith",  "14"],
    ["Wool", "Smith",  "15"]
])
```

三列分别是名字、姓氏、成绩。数组使用统一字符串类型，因此成绩也是字符串。

第一步：取出姓氏列并生成条件。

```
mask = data[:, 1] == "Smith"
```

第二步：选择匹配行的成绩列。

```
scores = data[mask, 2]
```

得到 `["12", "13", "14", "15"]`。

第三步：转换成整数。

```
scores = scores.astype(int)
```

第四步：计算统计量。

```
avg = scores.mean()
spread = scores.std(ddof=0)

print(avg)
print(spread)
```

输出：

```
13.5
```

```
1.118033988749895
```

可以合并筛选与转换：

```
scores = data[data[:, 1] == "Smith", 2].astype(int)
```

但初学时分步写，更容易检查每一步。

20.2 模拟题型：创建与二维切片

题意：创建 1 到 20，排列成 4 行、5 列，取行位置 1 到 3、列位置 1 到 3，均包含位置 3。

```
arr = np.arange(1, 21).reshape(4, 5)
slice_arr = arr[1:4, 1:4]
```

结果：

```
[[ 7  8  9]
 [12 13 14]
 [17 18 19]]
```

检查点：

- arange 要包含 20，所以终点为 21。
- 切片要包含位置 3，所以右端写 4。
- 行列位置从 0 开始。
- 结果为 3 行、3 列。

20.3 模拟题型：利润、平均值条件与最大值位置

```
unit_price = np.array([12.0, 20.0, 30.0])
unit_cost = np.array([7.0, 14.0, 18.0])
quantity = np.array([2, 3, 4])

profit = (unit_price - unit_cost) * quantity
above_mean_idx = np.where(profit > profit.mean())[0]
best_idx = int(np.argmax(profit))
```

结果：

```
profit: [10. 18. 48.]
above_mean_idx: [2]
best_idx: 2
```

这里的平均利润约为 25.3333，只有位置 2 的利润严格大于平均值。

关键点：分清题目要“值”还是“位置” 。`profit[profit > profit.mean()]` 返回利润值；`np.where(...)[0]` 返回位置。

20.4 模拟题型：广播后逐行统计

```
matrix = np.array([
    [2.0, 4.0, 6.0, 8.0],
    [1.0, 3.0, 5.0, 7.0],
    [10.0, 20.0, 30.0, 40.0]
])

offset = np.array([1.0, 10.0, 100.0, 1000.0])

adjusted = matrix + offset
row_mean = adjusted.mean(axis=1)
```

矩阵有 3 行、4 列，偏移量有 4 个元素，所以每行加同一个偏移向量。

调整后的数组：

```
[[   3.   14.  106. 1008.]
 [   2.   13.  105. 1007.]
 [  11.   30.  130. 1040.]]
```

逐行均值：

```
[282.75 281.75 302.75]
```

21 查阅索引：从需求找到函数

下面是导航，不代替前文解释。不知道某个函数的参数时，可以跳到对应章节，或使用 `help()` 查看。

需求	写法
从列表创建数组	`np.array(data)`
指定类型创建	`np.array(data, dtype=int)`
维数、形状、元素数	`a.ndim`、`a.shape`、`a.size`
查看元素类型	`a.dtype`
转换元素类型	`a.astype(float)`
指定步长生成序列	`np.arange(start, stop, step)`
指定数量生成等距点	`np.linspace(start, stop, num)`
全零、全一数组	`np.zeros(shape)`、`np.ones(shape)`

需求	写法
未初始化的数组	`np.empty(shape)`
改变形状	`a.reshape(rows, cols)`
自动推算一维尺寸	`a.reshape(rows, -1)`
二维转置	`a.T`
独立的一维副本	`a.flatten()`
尽量共享内存地展平	`a.ravel()`
行列定位	`a[row, col]`
行列切片	`a[row_start:row_stop, col_start:col_stop]`
步长切片、倒序	`a[::2]`、`a[::-1]`
多个位置	`a[[0, 2, 4]]`
行列交叉子矩阵	`a[np.ix_(rows, cols)]`
逐元素乘法	`a * b`
矩阵乘法	`A @ B`
求和、平均、最小、最大	`a.sum()`、`a.mean()`、`a.min()`、`a.max()`
逐列统计	`a.mean(axis=0)`
逐行统计	`a.mean(axis=1)`
保留统计轴	`a.sum(axis=1, keepdims=True)`
标准差	`a.std(ddof=0)`
中位数、分位数	`np.median(a)`、`np.quantile(a, q)`
累计和	`np.cumsum(a)`
条件筛选	`a[a > x]`
多条件筛选	`a[(a >= low) & (a < high)]`
按条件生成值	`np.where(cond, x, y)`
一维条件位置	`np.where(cond)[0]`
最大、最小值位置	`np.argmax(a)`、`np.argmin(a)`
至少一个、全部成立	`np.any(cond)`、`np.all(cond)`
满足条件的个数	`cond.sum()`
新数组排序	`np.sort(a)`
原地排序	`a.sort()`
排序下标	`np.argsort(a)`
独立副本	`a.copy()`
去重与计数	`np.unique(a, return_counts=True)`
交集、并集、差集	`np.intersect1d(a,b)`、`np.union1d(a,b)`、`np.setdiff1d(a,b)`
判断缺失	`np.isnan(a)`

需求	写法
忽略缺失的平均与求和	`np.nanmean(a)`、`np.nansum(a)`
插入、追加、删除	`np.insert(a, i, value)`、`np.append(a, values)`、`np.delete(a, i)`
拼接	`np.concatenate([a, b], axis=0)`
平方根、绝对值	`np.sqrt(a)`、`np.abs(a)`
指数、自然对数	`np.exp(a)`、`np.log(a)`
标准正态随机数组	`np.random.randn(rows, cols)`
随机生成器	`rng = np.random.default_rng(seed)`
随机整数	`rng.integers(low, high, size=shape)`
保存、读取数组	`np.save(path, a)`、`np.load(path)`
保存、读取数值文本	`np.savetxt(...)`、`np.loadtxt(...)`
查询帮助	`help(np.arange)`

22 自测练习与参考答案

22.1 练习

先独立写代码，遇到忘记的函数可以查前文。查阅后能正确使用，也是需要训练的能力。

1. 创建 1 到 12，排列成 3 行、4 列，保存到 a。取前两行、最后两列，保存到 sub。
2. 对 a 计算每行总和、每列平均值。
3. 给 a 的四列分别加 10、20、30、40。
4. 从 `[12,35,8,50,20]` 中选出大于等于 20 且小于 50 的值。
5. 从 `[55,80,60,42]` 中找出不及格成绩的下标；再用 where 把不及格改成 0。
6. 把 `[30,10,50,20]` 降序排列，同时保留原数组。
7. 从 `[10,20,30,40]` 取前两个元素，创建独立副本，再把副本第一个元素改成 99。
8. 统计 `[18,20,18,21,20,18]` 中不同年龄的次数。
9. 计算 `[10,NaN,30,NaN]` 忽略缺失后的均值。
10. 把 a 保存成 `practice.npy`，然后读回到 loaded。

22.2 参考答案

```
import numpy as np

# 1
```

```
a = np.arange(1, 13).reshape(3, 4)
sub = a[:2, -2:]

# 2
row_total = a.sum(axis=1)
column_mean = a.mean(axis=0)

# 3
adjusted = a + np.array([10, 20, 30, 40])

# 4
prices = np.array([12, 35, 8, 50, 20])
selected = prices[(prices >= 20) & (prices < 50)]

# 5
scores = np.array([55, 80, 60, 42])
failed_idx = np.where(scores < 60)[0]
adjusted_scores = np.where(scores < 60, 0, scores)

# 6
values = np.array([30, 10, 50, 20])
descending = np.sort(values)[::-1]

# 7
original = np.array([10, 20, 30, 40])
independent = original[:2].copy()
independent[0] = 99

# 8
ages = np.array([18, 20, 18, 21, 20, 18])
unique_ages, counts = np.unique(ages, return_counts=True)

# 9
missing = np.array([10, np.nan, 30, np.nan])
avg = np.nanmean(missing)

# 10
np.save("practice.npy", a)
loaded = np.load("practice.npy")
```

部分结果检查：

```
sub:
[[3 4]
 [7 8]]
```

```
row_total: [10 26 42]
column_mean: [5. 6. 7. 8.]
selected: [35 20]
failed_idx: [0 3]
adjusted_scores: [ 0 80 60  0]
descending: [50 30 20 10]
original: [10 20 30 40]
independent: [99 20]
unique_ages: [18 20 21]
counts: [3 2 1]
avg: 20.0
```

材料范围说明

本讲义根据本次学习对话、课件 `lec02-DataExploration-1.pptx` 的 NumPy 部分，以及复习资料中的完整指南、考前速查、代码大全和相关模拟题整理。

课程学习顺序与函数解释以本次已经讲解的内容为主。涉及随机数、数据类型显示和浮点输出的例子，实际显示形式可能受环境影响。做题时应遵守题目要求的变量名、结果类型、边界和标准差口径，不要仅凭代码能够运行就判断答案完全正确。